***Mini Project #1: Audio↔Image Signal Synthesis using a Time-Frequency Representation***

*By: Carlos Rojas, Oscar Reyes, Emmanuel Ramirez*

## 1.0 Introduction

In this Mini-project, our goal is to be able to truncate an audio signal to our desired parameters, and then once it is a discrete time audio signal, we can create a time frequency image, which we can then recover back into an audio signal that loses some information. In order to visualize and confirm our data during the process of the conversions, we aim to use histogram figures and spectrograms.

In order to begin the process of truncating and making the audio signal downloaded a discrete time audio signal, we must use the STFT, or the discrete short-time Fourier transform, which is particularly useful for creating an isolated signal that can then be made into an image through a companding operation that limits the range of the results of the STFT. Then we can use a color mapping function to accurately represent the red, green, and blue intensities present in the image.

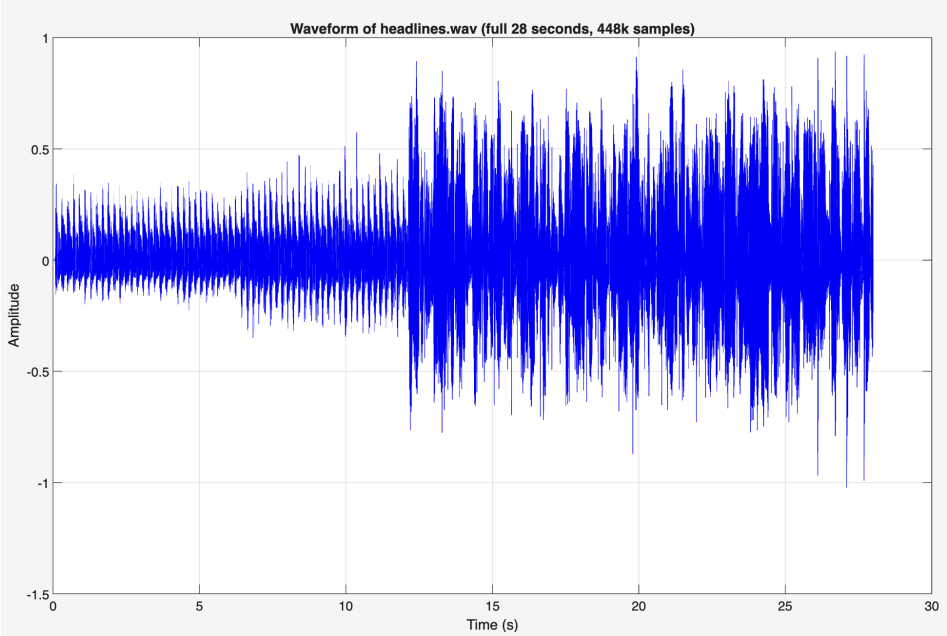
Lastly, once the image is formed, we can then use an inverse version of the color mapping function created that will effectively recover the audio signal that was present in the image, and we can designate it as the recovered audio signal for comparison in information loss.

## 2.0 Loading an Audio Signal

Once we have downloaded our desired audio signal, we resample the signal to 16,000 Hz, average the two stereo channels to convert it to mono, and truncate the signal to 448,000 samples.

```
[y, Fs] = audioread('headlines.wav');
y = resample(y, 16000, Fs);
y = mean(y, 2);
y = y(1:448000);
```

We then plotted the time-domain waveform of the entire 28 seconds. The plot shows amplitude fluctuations corresponding to speech/music content.




## 2.1 Computing the STFT and Inverse STFT

(a) We computed the STFT by applying a 448-point FFT to each column of the reshaped signal. This produced a 448×1000 complex-valued matrix S. (b) To verify correctness, we applied the inverse FFT and reshaped the result. The reconstructed signal matched the original closely, confirming reversibility.

```
S = fft(X, 448, 1);
Xhat = ifft(S, 448, 1, 'symmetric');
yhat = reshape(Xhat, [], 1);
soundsc(yhat, fs);
```

## 2.3 Companding

To compress the dynamic range of the STFT values, we applied the companding function:

$$F(x) = \text{sign}(x)\left((|x| + \epsilon)^p - \epsilon^p\right)$$

It's inverse F^-1 is used during decoding

```
function y = F(x)
    eps=0.1; power=0.4;
    y = sign(x) .* ((abs(x) + eps).^power - eps.^power);
end
```

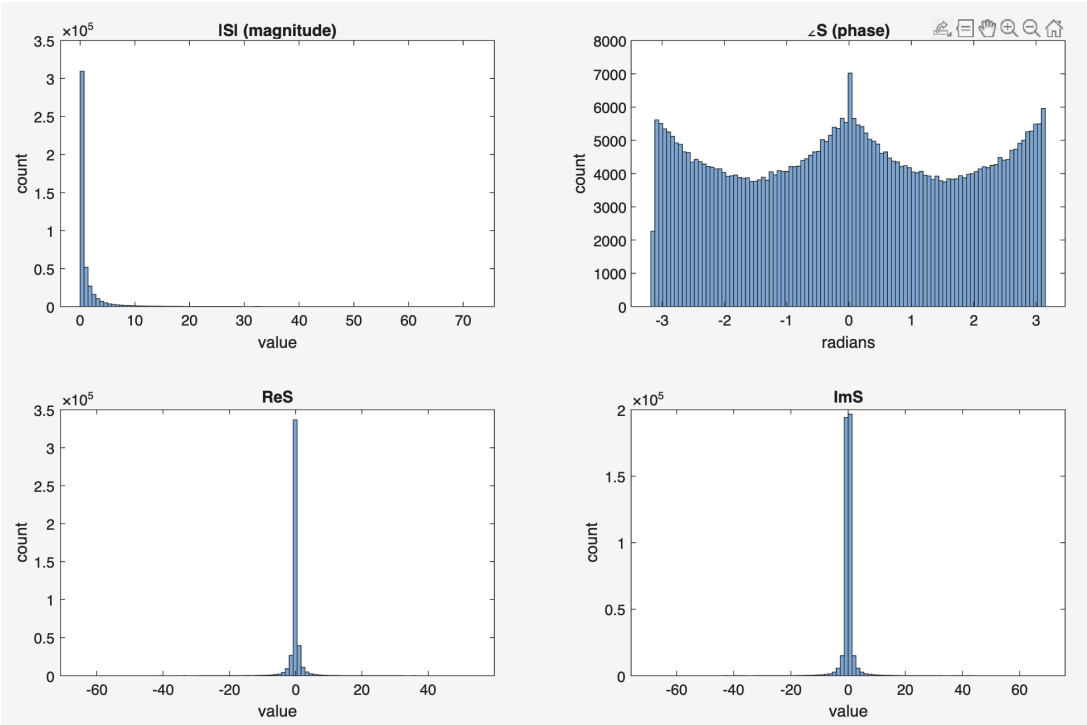
```
function x = Finv(y)
    eps=0.1; power=0.4;
    x = sign(y) .* ((abs(y) + eps.^power).^(1./power) - eps);
end
```

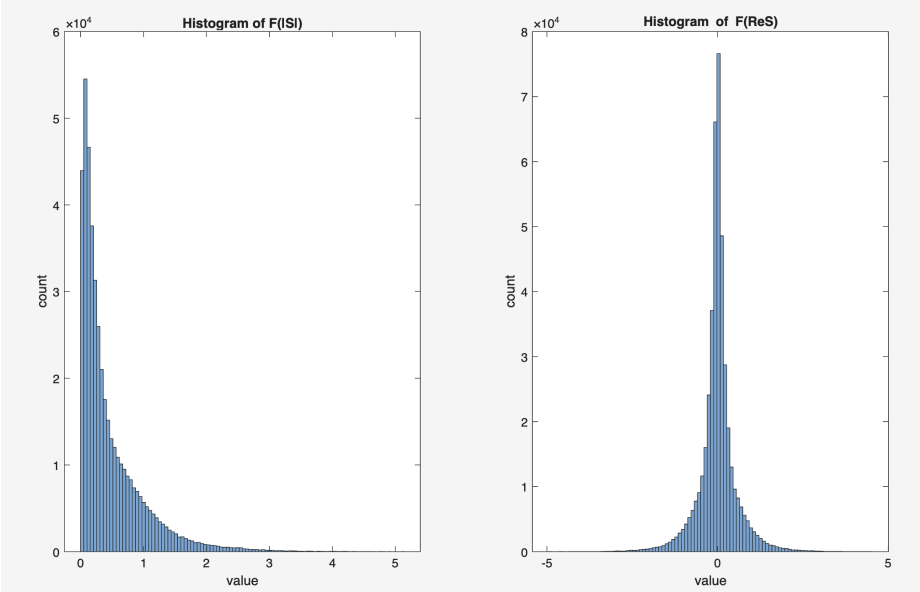
(a) Describe the shape of the distribution.
The magnitude distribution is heavily skewed toward smaller values, with a long tail extending to larger magnitudes. This is expected because most STFT coefficients are small, and only a few bins capture strong harmonics or energy bursts. The phase distribution is approximately uniform between $-\pi$ and $\pi$, but with peaks near 0 and $\pm\pi$, reflecting symmetry in sinusoidal components. The real and imaginary distributions are both centered at zero and exhibit a bell-shaped clustering, with most values close to zero and fewer values at larger magnitudes. This indicates that most frequency bins contain small oscillatory components.

We then compared histograms of the original values and their companded versions. Companding reduced skew and produced tighter distributions.

b) From the histograms, the ranges are as follows:

- The magnitude values span approximately 0 to 72.01.
- The phase values span the full range of $-\pi$ to $\pi$ (about -3.14 to 3.14).
- The real part values span roughly -64.39 to +53.56.
- The imaginary part values span roughly -68.39 to +68.39 as well.






(companded values)

## 2.4 Mapping Complex Values to a Color Image

a) We encode each complex value as follows:

- R channel: companded Re{S}, mapped to [0,1].
- G channel: companded Im{S}, mapped similarly.
- B channel: average of channels R and G.

```
[RGB,Ar,Ai] = RealImag2RGB(x);
```

```
function [RGB,Ar,Ai] = RealImag2RGB(S)  %option b
     FSreal = F(real(S));
     FSimag= F(imag(S));
     Ar = max(abs(FSreal(:)));
     Ai = max(abs(FSimag(:)));

     if Ar ==0    % Modify Ar or Ai if necessary to not divide by 0
         Ar = 1;
     end

     if Ai ==0
         Ai = 1;
     end

     R = (FSreal/Ar); % Normalize
     G = (FSimag/Ai);
     R = 0.5 + 0.5*(R); % Shift to get into [0,1] range
     G = 0.5 + 0.5*(G);
     B = (0.5*R) + (0.5*G);
     RGB = uint8(255*cat(3, R, G, B)); % represent as an RGB image;
end
```

```
inverse = RGB2Complex(RGB, Ar, Ai);
```

```
function S = RGB2Complex(RGB,Ar,Ai)
    R = double(RGB(:,:,1));
    G = double(RGB(:,:,2));
    B = double(RGB(:,:,3));
    R = R/255;
    G = G/255;
    FSreal = Ar*(2*R-1);
    FSimag = Ai*(2*G-1);
    S = Finv(FSreal) + 1j* Finv(FSimag);
end
```

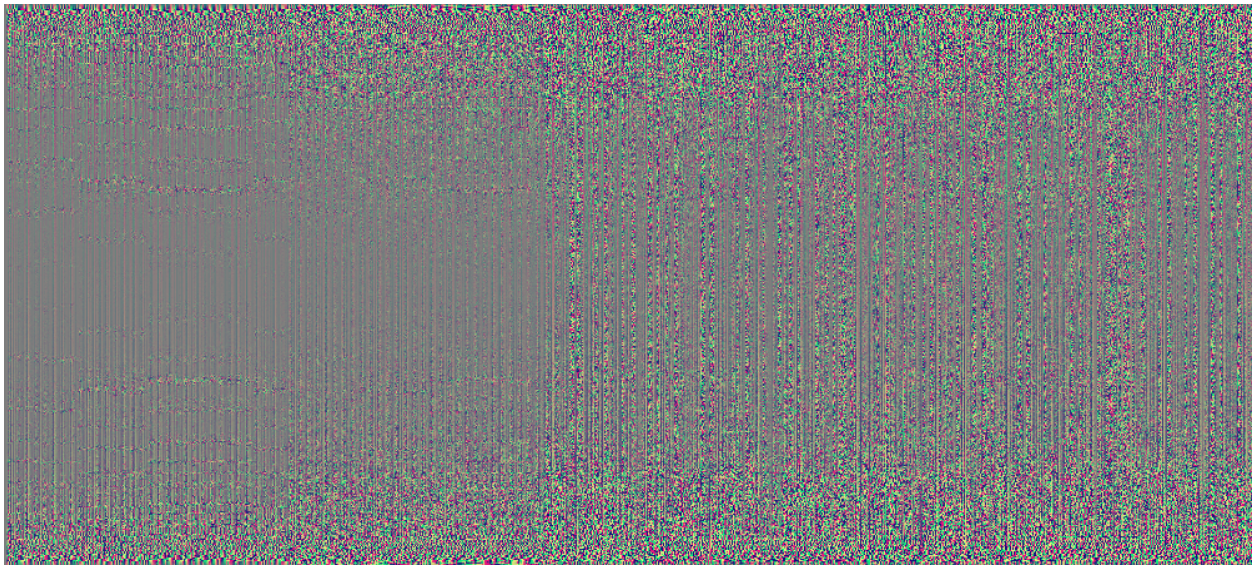


Image representation of audio signal

```
function [RGB,Ar,Ai] = RealImag2RGB(S)  %option b
     FSreal = F(real(S));
     FSimag= F(imag(S));
     Ar = max(abs(FSreal(:)));
     Ai = max(abs(FSimag(:)));

     if Ar ==0    % Modify Ar or Ai if necessary to not divide by 0
         Ar = 1;
     end

     if Ai ==0
         Ai = 1;
     end

     R = (FSreal/Ar); % Normalize
     G = (FSimag/Ai);
     R = 0.5 + 0.5*(R); % Shift to get into [0,1] range
     G = 0.5 + 0.5*(G);
     B = (0.5*R) + (0.5*G);
     RGB = uint8(255*cat(3, R, G, B)); % represent as an RGB image;
end
```

## 3.0 Recovering Audio from the Time-Frequency Image

In this section, we address the tasks required for recovering the audio signal from its image representation. First, we read the saved PNG image (S_realimag.png) back into MATLAB, which corresponds to Question 3.0(a). Once loaded, the inverse mapping was applied to convert the pixel intensities in the R and G channels back into the real and imaginary parts of the STFT coefficients. This step directly answers Question 3.0(b), as it reconstructs the matrix S[m,k] from its Real/Imag representation.

After recovering the STFT matrix, we performed an inverse FFT on each column to transform the data back into the time domain. Concatenating these columns produced the reconstructed signal, thereby answering Question 3.0(c). To evaluate the quality of the reconstruction, we computed the Peak Signal-to-Noise Ratio (PSNR) between the original truncated audio and the reconstructed version. The PSNR was calculated using the formula provided in the project handout:

$$\begin{aligned}\mathrm{PSNR}(x[n], y[n]) &= 20\log_{10}\left(\frac{(I_{\max} - I_{\min})^2}{\mathrm{MSE}(x[n], y[n])}\right) \\ &= 20\log_{10} 2 - 10\log_{10}\big(\mathrm{MSE}(x[n], y[n])\big) \\ &= -10\log_{10}\big(\mathrm{MSE}(x[n], y[n])\big) + 6.02 \text{ dB}\end{aligned}$$

This analysis completes Question 3.0(d). As expected, the direct IFFT (without image conversion) yielded a nearly perfect reconstruction with a very high PSNR. In contrast, the Real/Imag mapping introduced quantization noise and small distortions, leading to lower PSNR values. Nonetheless, the reconstructed signal remained clearly recognizable.

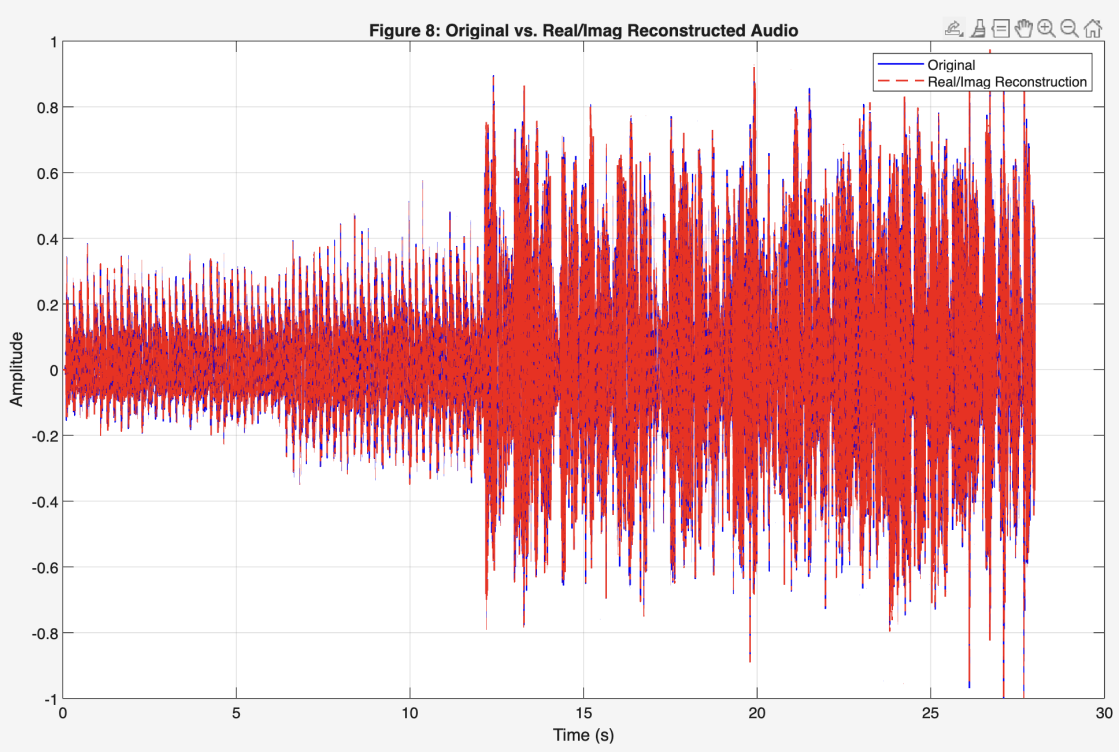
```
y_rec_ri = reshape(ifft(S_from_realimag, 448, 1, 'symmetric'), [], 1);

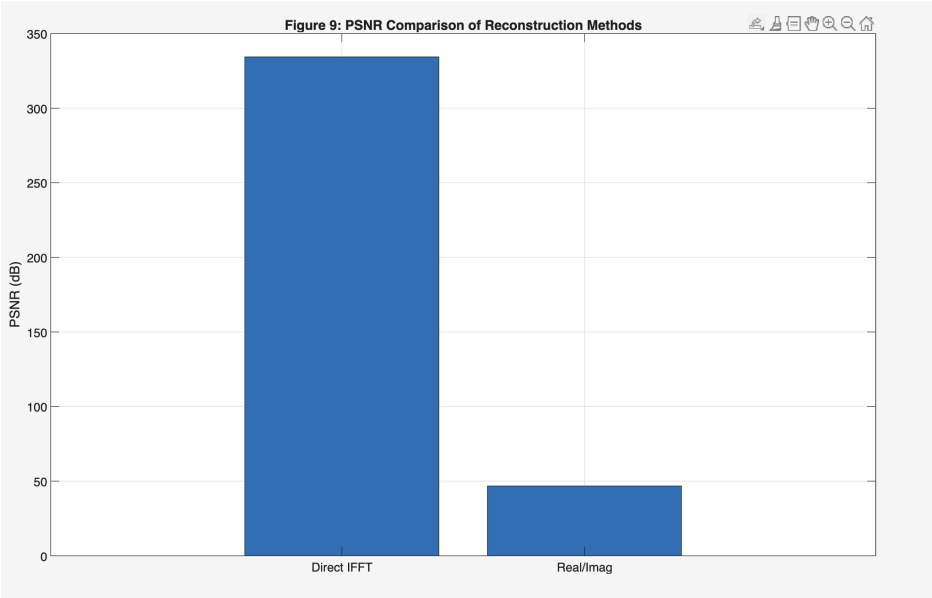
y_norm     = max(-1, min(1, y));
yhat_norm  = max(-1, min(1, yhat));
yrecri_norm= max(-1, min(1, y_rec_ri));

% ideal (no image step)
PSNR_ifft = -10*log10(mean((y_norm - yhat_norm).^2)) + 6.02;
% real/imag image path
PSNR_ri   = -10*log10(mean((y_norm - yrecri_norm).^2)) + 6.02;

fprintf('PSNR (direct): %.2f dB\n', PSNR_ifft);
fprintf('PSNR (Real/Imag image path):   %.2f dB\n', PSNR_ri);
```

```
PSNR (direct): 334.38 dB
PSNR (Real/Imag image path):   46.78 dB
```




(lower PSNR better)

## 4.0 Conclusion

After successfully being able to truncate our original audio, average the two channels to create a mono type of channel, create a Time-Frequency Image, and recover the original audio, there was only a slightly audible difference. When playing the reconstructed audio, some of the higher frequencies sound less sharp than the original audio. It is also important to note that we used option b for both the color mapping and the inverse mapping so that we could create the image and reconstruct the audio from the image.

*References:*

*ChatGPT Model 5*
*Mini project 1 assignment PDF- Prof. Brian Evans & Mr. Dan Jacobellis*
*Homework Help & Hints Web page- Prof. Brian Evans & Mr. Dan Jacobellis*
*https://a1017-86403320.cluster34.canvas-user-content.com/courses/1017~1424307/files/1017~86403320/course%20files/signals/homework/fall2025/miniproj1hints/mini_project_1.html*

*Both Matlab Code Source References*

```
% Read the audio file
[y, Fs] = audioread('headlines.wav');
q = 16000;
% Change sample to 16,000Hz
z = resample(y,q,Fs);
% Change from stereo to mono
z = mean(z,2);
%Corresponds to section 2.0
% Truncate to 448,000 samples (28 sec)
z = z(1:448000);
%Corresponds to section 2.1
|'
\≥z = reshape(z,[448,1000]);
x = fft(z);
a = ifft(x); %part a
a = reshape(a, [448000,1]); %part b
soundsc(a, q);
sMag = abs(x);
sPhase = angle(x);
sReal = real(x);
sImag = imag(x);
% plot the histograms
% Corresponds to section 2.3
figure; histogram(sMag(:));
figure; histogram(sPhase(:));
figure; histogram(sReal(:));
figure; histogram(sImag(:));
% Describe shapes of the distributions
% What is the range of each?
% Magnitude
range_mag = [min(sMag(:)), max(sMag(:))];
% Phase
range_phase = [min(sPhase(:)), max(sPhase(:))];
% Real part
range_real = [min(sReal(:)), max(sReal(:))];
% Imag part
range_imag = [min(sImag(:)), max(sImag(:))];
% Display results
fprintf('|S| range:      [%.2f, %.2f]\n', range_mag(1), range_mag(2));
fprintf('Phase range:    [%.2f, %.2f]\n', range_phase(1), range_phase(2));
fprintf('Real part range:[%.2f, %.2f]\n', range_real(1), range_real(2));
fprintf('Imag part range:[%.2f, %.2f]\n', range_imag(1), range_imag(2));
fMag = F(sMag);
fReal = F(sReal);
fImag = F(sImag);
```

```
figure; histogram(fMag(:)); %what is the range?
figure; histogram(fReal(:)); %what is the range?
%Corresponds to section 2.4
figure; imagesc(fMag);
figure; imagesc(fReal);
figure; imagesc(fImag);
figure; imagesc(sPhase);
[RGB,Ar,Ai] = RealImag2RGB(x);
figure; image(RGB); axis xy; title('Real/Imag to RGB');
imwrite(RGB,'realimag.png'); %Create image file
%Corresponds to section 3.0
RGB = imread('realimag.png');
inverse = RGB2Complex(RGB,Ar,Ai);
inverse  = RGB2Complex(RGB, Ar, Ai);
a_rec = ifft(inverse, [], 1);
a_rec = real(a_rec);
a_rec = a_rec(:);
sound(a_rec, q);
function [RGB,Ar,Ai] = RealImag2RGB(S)  % option b
FSreal = F(real(S));
FSimag= F(imag(S));
Ar = max(abs(FSreal(:)));
Ai = max(abs(FSimag(:)));
if Ar ==0    % Modify Ar or Ai if necessary to not divide by 0
    Ar = 1;
end
if Ai ==0
    Ai = 1;
end
R = (FSreal/Ar); % Normalize
G = (FSimag/Ai);
R = 0.5 + 0.5*(R); % Shift to get into [0,1] range
G = 0.5 + 0.5*(G);
B = (0.5*R) + (0.5*G);
RGB = uint8(255*cat(3, R, G, B)); % represent as an RGB image;
end
function y = F(x)
eps=0.1; power=0.4;
y = sign(x) .* ((abs(x) + eps).^power - eps.^power);
end
function x = Finv(y)
eps=0.1; power=0.4;
x = sign(y) .* ((abs(y) + eps.^power).^(1./power) - eps);
end
function S = RGB2Complex(RGB,Ar,Ai)
R = double(RGB(:,:,1));
G = double(RGB(:,:,2));
B = double(RGB(:,:,3));
R = R/255;
```

```
G = G/255;
FSreal = Ar*(2*R-1);
FSimag = Ai*(2*G-1);
S = Finv(FSreal) + 1j* Finv(FSimag);
end
```